



Database Operations

Practical Skills Module 4: Topic 6

[Visit our website](#)

Practical Skills Module 4: Topic 6

Topic Code	PM-04-PS05
Topic	Handle a Database in Python

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, the learner must be able to:

- PA0601 Interact with database
 - Establish a connection to database of choice
 - Create a cursor to communicate with the data
 - Manipulate the data using SQL (interact)
 - Insert
 - Select
 - Update
 - Delete
 - Commit the changes and make them permanent (commit), or
 - Rollback and abort manipulations thus returning the data to the state before the interactions occurred
 - Close the connection to the database
- PA0602 Browse the database

Applied Knowledge

- AK0601 Concept, definition and functions of intermediate Python programming principles

Internal Assessment Criteria

- IAC0601 Expected results with Python database handling are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-04



Practical task

Follow the facilitator's instructions to complete the following activities:

Save a screenshot as “Database” with the steps followed:

Scenario:

A retail business needs to manage its **product inventory** stored in a database. This includes adding new products, viewing existing records, updating stock quantities, deleting discontinued items, and ensuring the data integrity through commits or rollbacks. Additionally, the business wants to browse the database to display all available products.

Go to the [**DB Fiddle Online SQL Editor**](#). This is where you can write and test your SQL code. Once you are happy with your code, **paste it into a text file and save the file in your task folder as Student.txt.**

You will be working with a database table called *Student*. Follow the instructions below step by step.

Step 1: Create the Student Table

Write the SQL code to create a table named *Student*. Use the table structure provided below.

Note that *STU_NUM* must be set up as the **primary key**.

Table Structure:

Attribute Name	Data Type
STU_NUM	CHAR(6)

STU_SNAME VARCHAR(15)

STU_FNAME VARCHAR(15)

STU_INITIAL CHAR(1)

STU_STARTDATE DATE

COURSE_CODE CHAR(3)

PROJ_NUM INT(2)

Step 2: Insert the Following Records into the Student Table

Use `INSERT INTO` statements to add each of the following rows of data to the table:

STU_NUM	STU_SNAME	STU_FNAME	STU_INITIAL	STU_STARTDATE	COURSE_CODE	PROJ_NUM
01	Snow	Jon	E	2014-04-05	201	6
02	Stark	Arya	C	2017-07-12	305	11
03	Lannister	Jamie	C	2012-09-05	101	2
04	Lannister	Cercei	J	2012-09-05	101	2
05	Greyjoy	Theon	I	2015-12-09	402	14
06	Tyrell	Margaery	Y	2017-07-12	305	10
07	Baratheon	Tommen	R	2019-06-13	201	5

You may insert the rows one at a time, or all together using a single `INSERT INTO` statement with multiple `VALUES`.

Step 3: Retrieve Records by Course Code

Write the SQL code to return **all records where the COURSE_CODE is 305**.

Use a `SELECT` statement with a `WHERE` clause.

Step 4: Update a Specific Record

Write the SQL code to **change the COURSE_CODE to 304** for the student whose `STU_NUM` is '07'.

Use an `UPDATE` statement with a `WHERE` clause to target the correct row.

Step 5: Delete a Specific Record Using Multiple Conditions

Write the SQL code to **delete the row for Jamie Lannister**, using **all of the following conditions**:

- `STU_SNAME` is 'Lannister'
- `STU_FNAME` is 'Jamie'
- `STU_STARTDATE` is '2012-09-05'
- `COURSE_CODE` is '101'
- `PROJ_NUM` is 2

Use a `DELETE FROM` statement and combine the conditions using `AND`.

Step 6: Update Records Conditionally

Write the SQL code to **set the PROJ_NUM to 14** for all students who:

- **Started before** '2016-01-01'
- **AND** have a `COURSE_CODE` greater than or equal to 201

Use an `UPDATE` statement with logical comparisons.

Step 7: Delete the Table

Write the SQL code to **delete the entire Student table**.

Hint: Use the `DROP TABLE` statement.

Perform an UPDATE to temporarily change the PROJ_NUM of any student (e.g., change it to 99).

Then **ROLLBACK** the change so that the original PROJ_NUM is preserved.

After that, perform another **UPDATE** (e.g., set PROJ_NUM to 88) and **COMMIT** the change.

Finally, **SELECT** the relevant records to confirm that:

- The first change was not saved (rollback worked)
- The second change was saved (commit worked)



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
